

Construction Company Management System

Content of document

User Requirement Text	2
Use Case Diagram	4
Use Case Scenarios	5
Activity Diagrams	7
State Diagram	9
Class Diagram - Analytical	10
Class Diagram - Design	11
Design Decisions	12
Dynamic Analysis	14
Non-Functional Requirements	15
GUI	16

User Requirement Text

1. System Overview The architecture firm plans to develop a Construction Office Management System. The primary goal of this application is to support managers in overseeing bureaus, projects, customers, employees, and construction zones efficiently.

2. Bureau Management The system must store data regarding the firm's bureaus, including their specific location and contact email. Each bureau must specify the primary service it offers, which can include Urban Planning, Management, Sustainability, and Constructing.

3. Employee Management The system must track comprehensive employee data, including their name, surname, email, hire date, discharge date, minimum wage, income, and overall qualification.

- **Dynamic Roles:** The system must support dynamic and overlapping employee roles, recognizing that staff members can change roles or hold multiple titles simultaneously.
- **Specializations:** Employees can act as **Engineers** (requiring a specific specialty such as Mechanical, Electrical, or Plumbing), **Landscape Architects** (requiring a specific license number), or **Managers** (assigned a management tier: Junior, Middle, or Senior manager).
- Managers must be able to hire employees, view employee data, and manage their assignments.

4. Customer Management For clients, the system must store their name, surname, phone number, email, and VAT number. Managers must be able to add new customers to the system. Every project created in the system must belong to exactly one customer.

5. Project Management Managers must be able to create projects, view a list of all projects, and update project statuses. Each project requires a name, budget, current state, start date, and realization date.

Projects must be categorized by two mutually exclusive (disjoint) classifications:

- **Target Audience:** A project is either a **Business** project (requiring the name of the company) or a **Residential** project (requiring a resident capacity).
- **Discipline:** A project is either a **Landscape Project** (requiring the total land area) or an **Interior Project** (requiring design details).

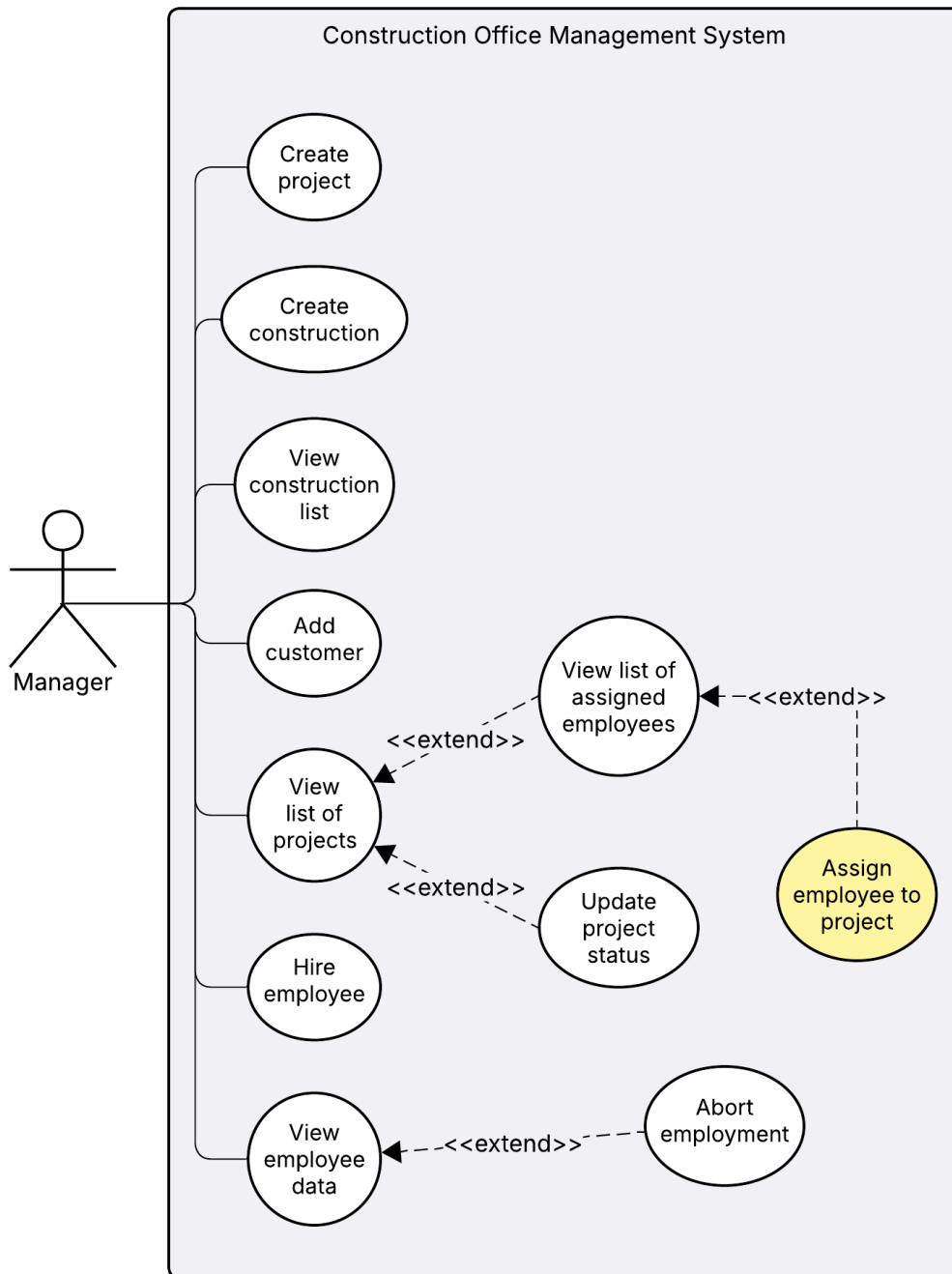
6. Construction Management Projects may require physical constructions. Managers must be able to create constructions and view a list of them.

- Constructions cannot exist independently; they must be tied to a specific project.
- Each construction within a specific project is uniquely identified by its **Construction ZoneName**.
- The system must track the specific location, date of start, and date of end for each construction zone.

7. Employee Project Assignments Managers must be able to select a project to view a list of all currently assigned employees and their roles. Managers must also be able to assign employees to projects.

- **Assignment Details:** Each assignment (Project Role) must record a role description, a start date, and an end date.
- **Validation Rules:** Employees can be assigned to a project multiple times. However, the system must perform a validation check during assignment: an employee *cannot* be assigned to the exact same role within an overlapping time interval. If this occurs, the system must block the action and display a pop-up error message. The assignment is only saved and added to the employee's history if the manager confirms the valid role and time interval.

Use Case Diagram



Use Case Scenarios

Construction Office Management System UC Scenario - View list of assigned employees

Actor : Manager

Purpose and context: The actor may display the list of assigned employees for selected project.

Assumptions:

- The employee and project data are complete and up-to-date.
- Each project has at least one employee already assigned to the project.

Pre-conditions:

- Project has been created.
- The list of current projects is displayed on the interface.

Initiating business event: Click on one project from the list of projects.

Basic Flow of Events:

- 1- Manager clicks on one of the projects from the displayed list of projects.
- 2- System displays the list of assigned employees to selected project along with their role.

Post-conditions:

- The list of assigned employees is presented on the screen.

Actor: Manager

Purpose and context: The actor may assign employee to a project

Assumptions:

- The employee and project data are complete and up-to-date.
- The manager has the authorization to assign employees to projects.
- A project can have multiple employees assigned with different roles and at the same time frame, or different time frames with the same role.

Pre-conditions:

- Project has been created.
- Manager is viewing the list of currently assigned employees with their role details.

Initiating business event: Click on "Assign Employee" button in the list of employees assigned for clicked project.

Basic Flow of Events:

- 1- Manager clicks on "Assign Employee" button for the selected project.
- 2- System displays employee selection lists.
- 3- Manager selects an employee from the list.
- 4- System displays Project Role form.
- 5- Actor enters role of the employee for that specific project.
- 6- Actor sets start and end date of assigned role.
- 7- Actor clicks on set assignment button.
- 8- System displays pop-up message "Are you sure?" to confirm.
- 9- Actor clicks on "yes"
- 10- System saves the assignment to the project and updates the employee's assignment history.
- 11- System displays the employees list for selected project.

Alternative Flow of Events:

• **Invalid data on project role configuration**

• **Invalid role for given time interval**

- 7a1- System displays pop-up error message "Requested role already assigned for an overlapping time interval"
- 7a2- Returns to step 4

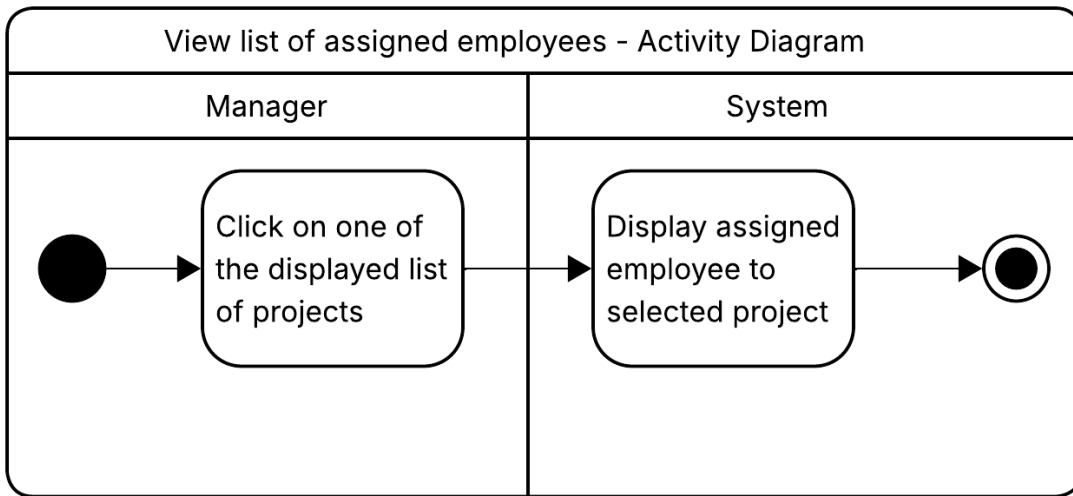
Actor clicks on "Cancel"

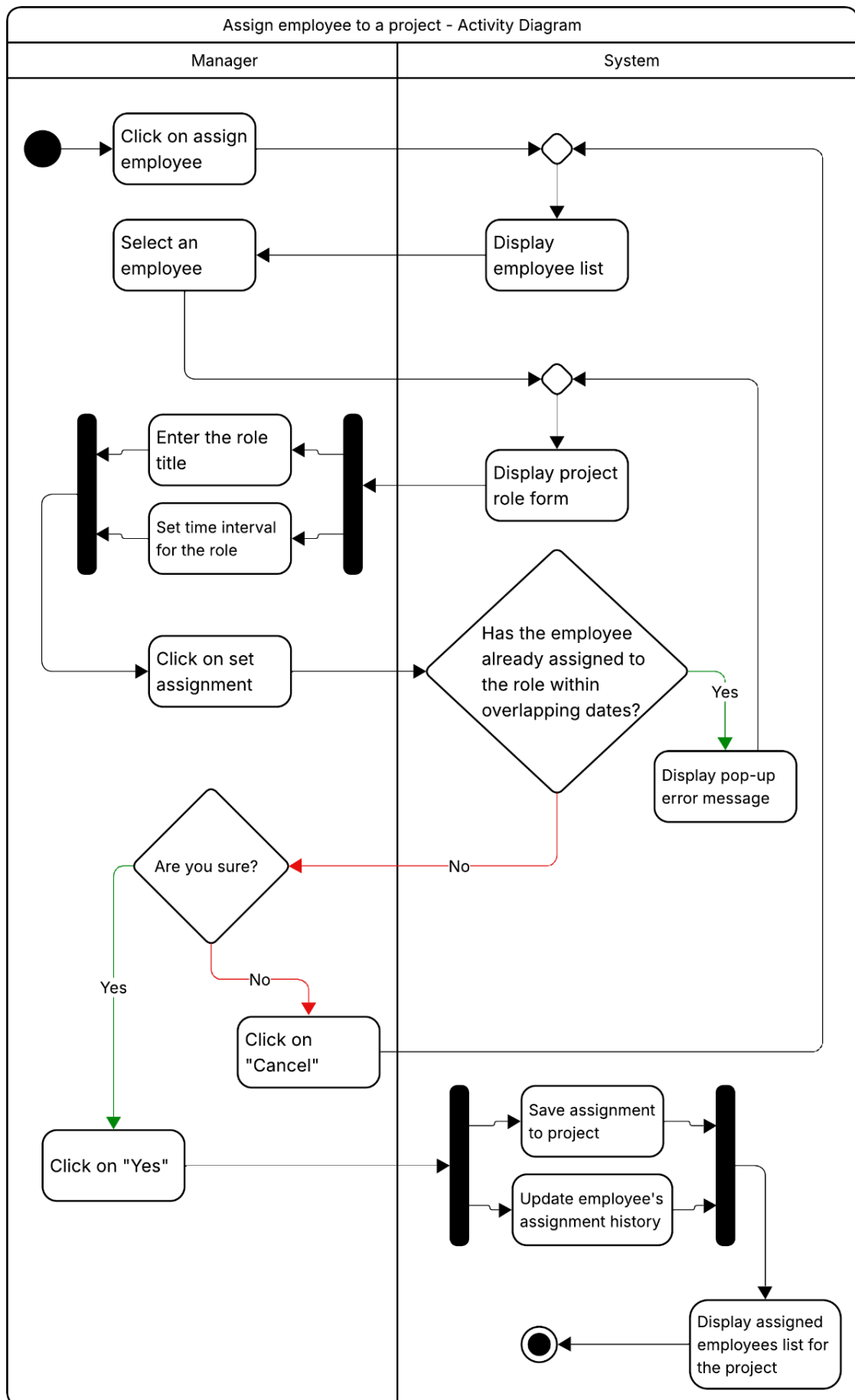
- 9a1- System turns back to step 2.

Post-conditions:

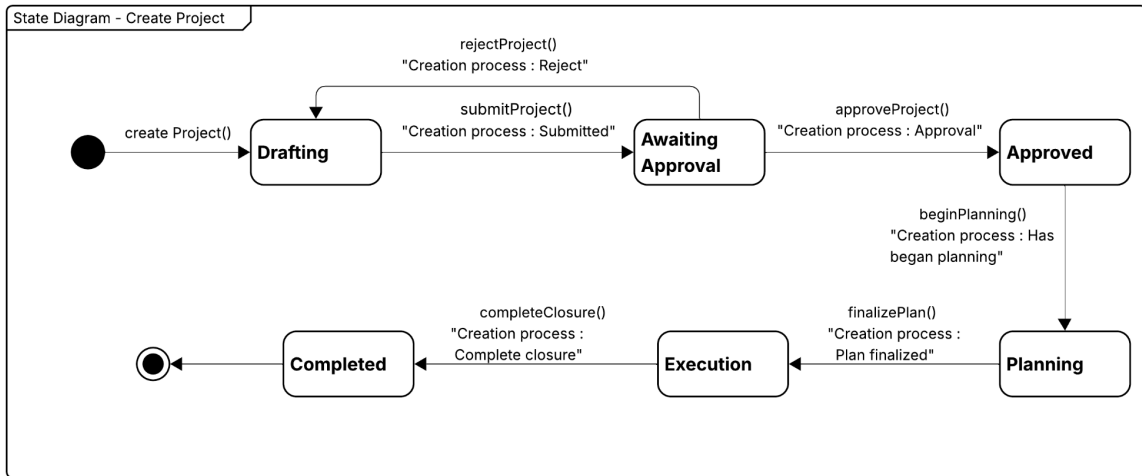
- The employee is successfully assigned to the project.
- The assignment details are stored in the system.

Activity Diagrams

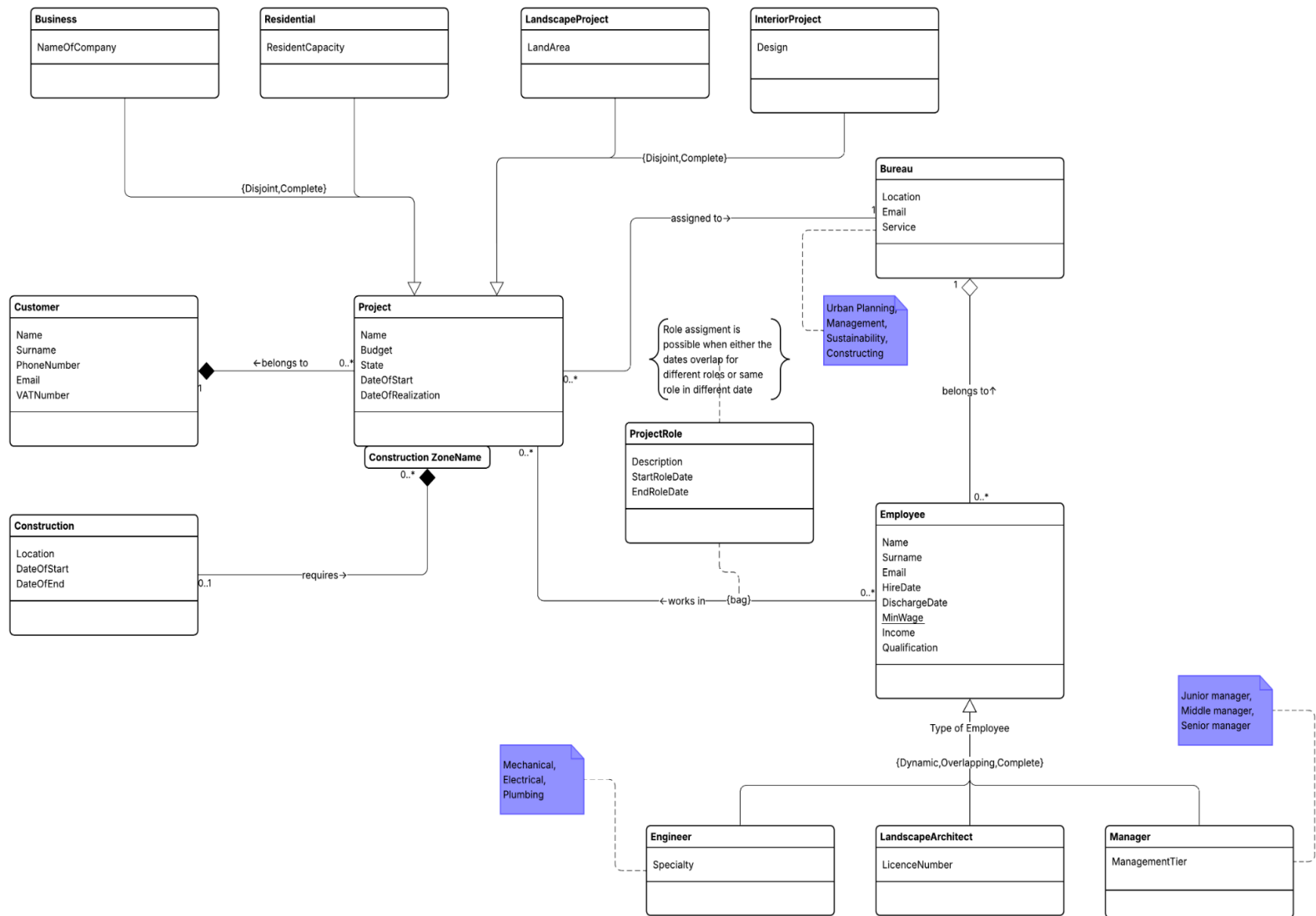




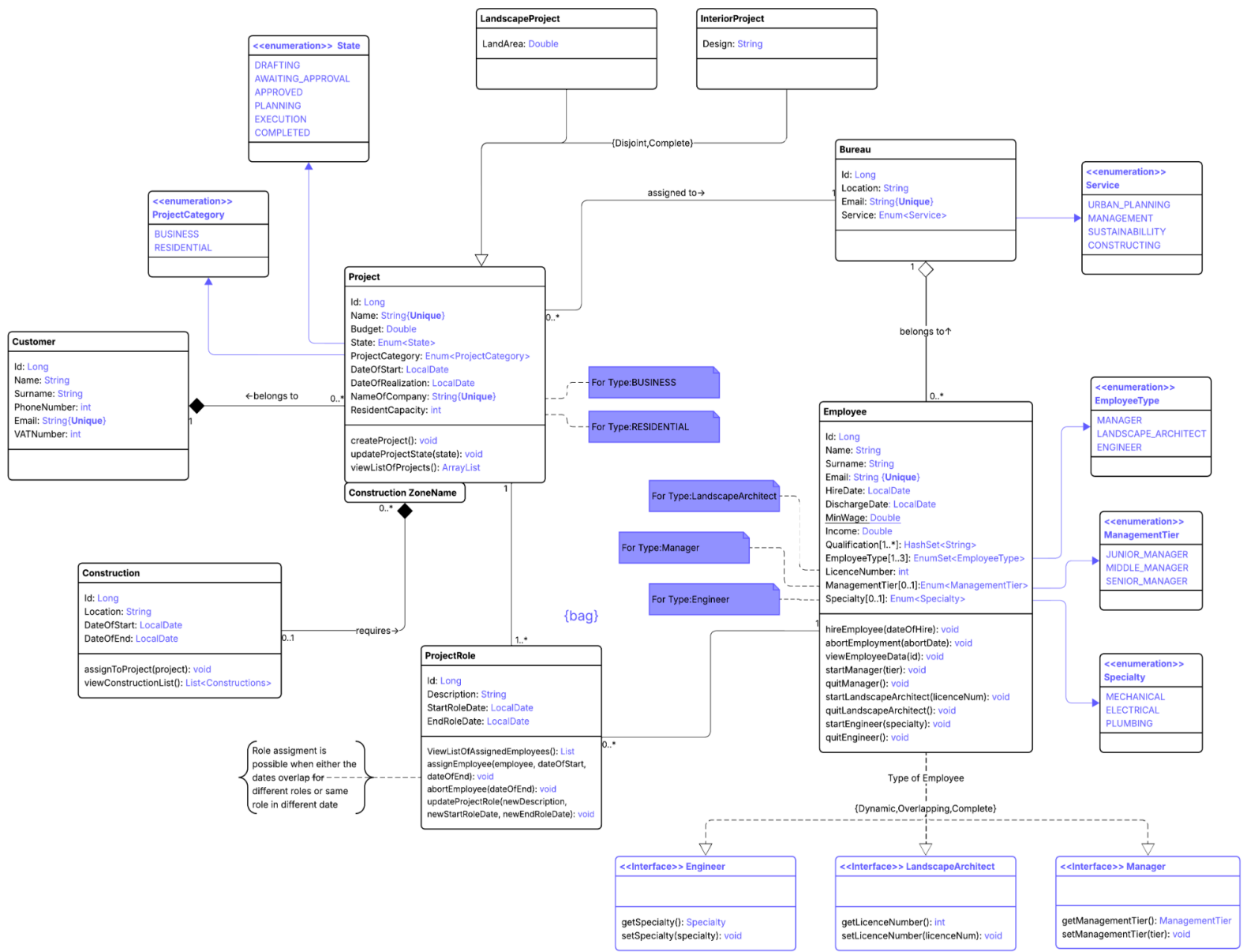
State Diagram



Class Diagram - Analytical



Class Diagram - Design



Design Decisions

The design class diagram utilizes standard Java constructs to approximate UML features that have no direct language equivalent. The final implementation targets a desktop application built with the Spring Framework, Swing (for the GUI), Lombok (for boilerplate reduction), and Hibernate (as the JPA provider) backed by an H2 in-memory database.

Below, we detail the strategies used to transform the UML associations, validations, and complex inheritance patterns into viable Java code.

1. Attribute Validations

To enforce fundamental data-consistency rules (e.g., required fields, valid dates, positive numbers), the implementation relies heavily on Jakarta Validation annotations.

- **Mandatory Fields:** Attributes such as `Name`, `Surname`, and `Email` in the `Customer` and `Employee` classes are annotated with `@NotBlank`.
- **Numerical Constraints:** `Budget` (in `Project`) and `MinWage` (in `Employee`) are annotated with `@Min` to prevent negative values.
- **Date Constraints:** Fields like `DateOfStart` are validated with `@FutureOrPresent` to prevent backdating where inappropriate.

2. Tagged Values (Enums)

Several UML attributes are constrained to a fixed set of valid values (e.g., `ManagementTier`, `Specialty`, `Service`, `State`). In Java, these are implemented as strict `enum` types to guarantee compile-time safety.

- Each enum field in the entities is annotated with `@Enumerated(EnumType.STRING)` so that Hibernate persists the literal string value (e.g., `"SENIOR_MANAGER"`) in the database rather than a brittle integer ordinal.

3. Qualified Associations (Maps)

The UML model uses a qualified association between `Project` and `Construction` based on the `ZoneName`. Because a qualifier guarantees a unique lookup within that association, this is implemented in Java using a `Map` rather than a standard `List`.

- Inside the `Project` entity, the association is mapped as: `@OneToMany(mappedBy = "project") @MapKey(name = "zoneName") private Map<String, Construction> constructions = new HashMap<>();`
- This provides direct, $O(1)$ access to a specific construction phase via its string name.

4. Association With Attributes (Association Classes)

UML permits "association classes" (relationships that hold their own data). Java does not support this directly.

- We resolve this by creating a dedicated intermediate entity: **ProjectRole**.
- **ProjectRole** contains its own fields (**Description**, **StartRoleDate**, **EndRoleDate**) and holds **@ManyToOne** foreign keys to exactly one **Employee** and exactly one **Project**.
- Business logic inside the service layer handles the instantiation of these roles and ensures no employee is assigned to the same role during an overlapping time interval.

5. Overlapping & Dynamic Inheritance

The UML model requires employees to dynamically change or hold multiple roles simultaneously (Manager, Engineer, Landscape Architect). Java prohibits a class from inheriting from multiple parents and prevents changing an object's type at runtime. To resolve this, we use a **Flattened Hierarchy Strategy**:

- **Single Entity**: We maintain one comprehensive **Employee** class containing all possible fields for all roles (e.g., **LicenceNumber**, **Specialty**, **ManagementTier**). These role-specific fields are marked as nullable.
- **Interfaces**: The roles are defined as Java interfaces (**Engineer**, **Manager**, **LandscapeArchitect**.) that the **Employee** class implements.
- **State Tracking**: The active roles are tracked using an **EnumSet<EmployeeType> employeeTypes**.
- **Dynamic Methods**: Role-switching is handled via methods like **startManager(ManagementTier tier)** and **quitManager()**, **startLandscapeArchitect(licenceNum)** and **quitLandscapeArchitect()**, **startEngineer(specialty)** and **quitEngineer()**. Activating a role populates the specific fields and adds the type to the **EnumSet**; quitting nullifies the fields and removes the type.

6. Disjoint & Complete Inheritance (Multi-Aspect)

The **Project** class is categorized by two distinct aspects: Target Audience (**Business**, **Residential**) and Discipline (**LandscapeProject**, **InteriorProject**). Since Java does not support multiple inheritance, creating standard subclasses for all combinations (e.g., **BusinessLandscapeProject**) would result in class explosion.

- **Primary Aspect via Subclassing**: The Discipline aspect is handled using JPA inheritance. **Project** acts as the base class utilizing **@Inheritance(strategy = InheritanceType.JOINED)**. **LandscapeProject** and **InteriorProject** exist as explicit subclasses extending **Project** in the database.
- **Secondary Aspect via Composition/Enums**: The Target Audience aspect is handled via the **ProjectCategory** enum (**BUSINESS**, **RESIDENTIAL**) stored on the base **Project** class, alongside nullable fields (**NameOfCompany**, **ResidentCapacity**). The service layer strictly enforces that a **BUSINESS** project must have a company name, and a **RESIDENTIAL** project must have a capacity.

Dynamic Analysis

During the design of the "Create Project" state diagram, it made sense to track and control the lifecycle of each project in the system. A project cannot jump from creation directly to execution because it must pass through a strict sequence of approvals and planning phases.

To handle this dynamic behavior, the implementation introduces a **State** enumeration (as defined in the Design Class Diagram) to represent the project's current status at any given time. The **State** enum defines the following precise lifecycle states:

- **DRAFTING**
- **AWAITING_APPROVAL**
- **APPROVED**
- **PLANNING**
- **EXECUTION**
- **COMPLETED**

While the UML state diagram conceptually labels every transition trigger as **createProject()** with varying conditions, the actual Java implementation delegates these transitions to distinct, secure methods within the Service Layer. This ensures that the state machine is encapsulated and cannot be bypassed.

The dynamic transitions are implemented via the following service methods:

- **Initialization:** **createProject(String name, Double budget, ProjectCategory category)** → Sets initial state to **DRAFTING**
- **Submission & Approval Flow:** **submitProject(Long projectId)** → Transitions from **DRAFTING** to **AWAITING_APPROVAL**
rejectProject(Long projectId, String reason) → Transitions from **AWAITING_APPROVAL** back to **DRAFTING**
approveProject(Long projectId) → Transitions from **AWAITING_APPROVAL** to **APPROVED**
- **Execution Flow:** **beginPlanning(Long projectId)** → Transitions from **APPROVED** to **PLANNING**
finalizePlan(Long projectId) → Transitions from **PLANNING** to **EXECUTION**
completeClosure(Long projectId) → Transitions from **EXECUTION** to **COMPLETED**

By placing these transition methods in the service layer rather than the entity itself, the system can dynamically enforce security rules (e.g., verifying that the user calling **approveProject()** possesses the **Manager** role) before allowing the state to change.

Those methods are not included in the design diagram as they are in the service layer.

Non-Functional Requirements

1. Technology Stack

- The system must be a desktop application built using the Spring Framework and Java Swing for the user interface.
- The application must use an H2 in-memory database to store data easily without requiring external server setup.
- The code will use Lombok to automatically generate basic methods (like getters and setters) to keep the files clean and simple.

2. Basic Security & Access

- The system must ensure that only users with the "Manager" role can approve projects or change project states.
- In the code, this will be handled by a simple role check (e.g., checking the user's role before executing the state change) rather than a complex enterprise security configuration.

3. Data Rules

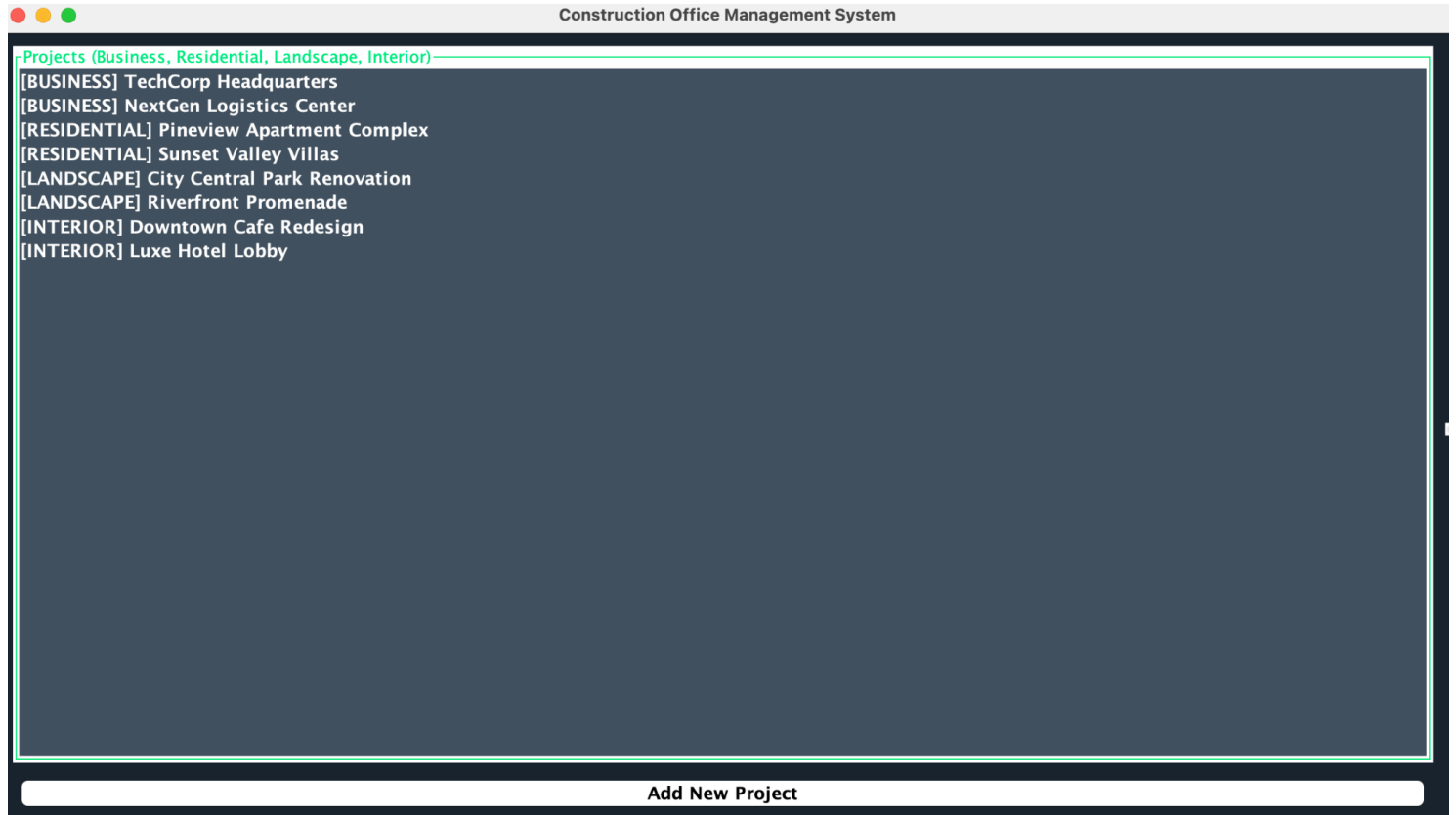
- The system must not allow blank fields for critical data, such as Employee and Customer names.
- The system must not allow negative numbers for numerical fields like the Project Budget or Employee Wage.
- The system must check dates to ensure an employee is not assigned to the same role during an overlapping time interval.

4. Usability & User Interface

- The interface must show a simple confirmation pop-up (e.g., "Are you sure?") before saving important data like an employee assignment.
- If a user types in wrong data (like a negative budget or overlapping dates), the system must not crash. Instead, it must show a clear pop-up error message telling the user what went wrong.

GUI

When the program starts, It displays a list of different projects.



Then the manager chooses one project from the list.

Construction Office Management System

Projects (Business, Residential, Landscape, Interior)

[BUSINESS] TechCorp Headquarters

[BUSINESS] NextGen Logistics Center

[RESIDENTIAL] Pineview Apartment Complex

[RESIDENTIAL] Sunset Valley Villas

[LANDSCAPE] City Central Park Renovation

[LANDSCAPE] Riverfront Promenade

[INTERIOR] Downtown Cafe Redesign

[INTERIOR] Luxe Hotel Lobby

Employees Assigned to Selected Project

Employee Name & Qualification	Description	StartRoleDate	EndRoleDate
John Doe – Senior Manager	Client Liaison	2024-09-01	2024-12-31
Karen Hill – Electrical Engineer	Lighting Consultant	2024-09-15	2024-11-15
Bob Johnson – Electrical Engineer	Power Grid Lead	2024-09-15	2024-12-01

Add New Project

Assign Employee to Project

The system displays all assigned employees and their role details. To assign a new employee, the manager clicks on the “Assign Employee to Project” button.

Assign Employee to: [INTERIOR] Downtown Cafe Redesign

Select Employee:

Select Qualification:

Role Description:

StartRoleDate (YYYY-MM-DD):

EndRoleDate (YYYY-MM-DD):

✓ John Doe

Alice Smith

Bob Johnson

Clara Woods

David Miller

Eva Stone

Frank Ocean

Grace Hopper

Henry Ford

Ivy Green

Jack Ryan

Karen Hill

2025-10-31

Save Assignment

Cancel

Because of bag constraint the system can display already assigned employees in this list, but we can't assign the same employee with same role type in the overlapping time interval, however, we can assign with a new role or with the same role in different time intervals.

Assign Employee to: [INTERIOR] Downtown Cafe Redesign

Select Employee: John Doe

Select Qualification: Senior Manager

Role Description: Site Supervisor

StartRoleDate (YYYY-MM-DD): 2024-05-01

EndRoleDate (YYYY-MM-DD): 2025-10-31

Save Assignment Cancel

Some employees can have more than one Qualification so we can choose for selected employee different Qualification.

Assign Employee to: [INTERIOR] Downtown Cafe Redesign

Select Employee: John Doe

Select Qualification: Senior Manager
✓ Electrical Engineer

Role Description: Site Supervisor

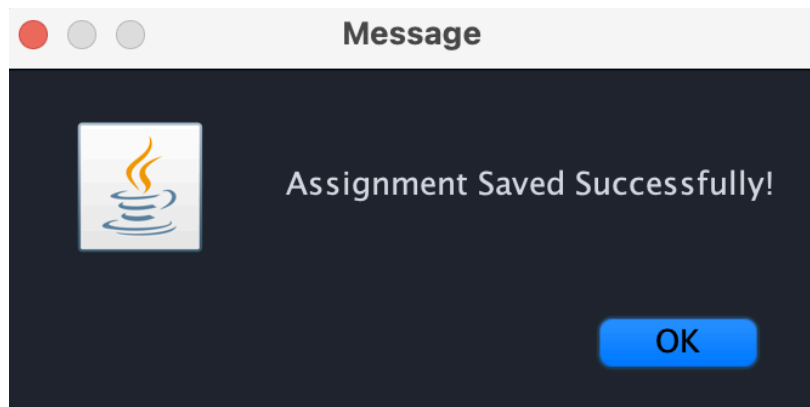
StartRoleDate (YYYY-MM-DD): 2024-05-01

EndRoleDate (YYYY-MM-DD): 2025-10-31

Save Assignment Cancel

After adding the data we need, we click “Save Assignment” button

This data system allow this assignment because employee already assigned as senior manager but not as electrical engineer.



But if we try to add the same Qualification and within the same time interval, we will get an error message and the system turns back to the “Assign Employee” form.

A screenshot showing two overlapping macOS-style windows. The top window is titled "Assign Employee to: [INTERIOR] Downtown Cafe Redesign". It contains several form fields: "Select Employee:" with a dropdown menu showing "John Doe"; "Select Qualification:" with a dropdown menu showing "Senior Manager"; "Role Description:" with a text field containing "Site Supervisor"; "StartRoleDate (YYYY-MM-DD):" with a text field containing "2024-05-01"; and "EndRoleDate (YYYY-MM-DD):" with a text field containing "2025-10-31". At the bottom are two buttons: "Save Assignment" and "Cancel". The bottom window is titled "Validation Error" and has a dark blue background. It features a red octagonal stop sign icon with a white exclamation mark and a small blue flame icon. The text "Requested role already assigned for an overlapping time interval!" is displayed in white. A blue "OK" button is at the bottom right.

After successfully assignment, the new assigned employee will be visible in the assigned employees list.

Construction Office Management System

Projects (Business, Residential, Landscape, Interior)

[BUSINESS] TechCorp Headquarters

[BUSINESS] NextGen Logistics Center

[RESIDENTIAL] Pineview Apartment Complex

[RESIDENTIAL] Sunset Valley Villas

[LANDSCAPE] City Central Park Renovation

[LANDSCAPE] Riverfront Promenade

[INTERIOR] Downtown Cafe Redesign

[INTERIOR] Luxe Hotel Lobby

Employees Assigned to Selected Project

Employee Name & Qualification	Description	StartRoleDate	EndRoleDate
John Doe - Senior Manager	Client Liaison	2024-09-01	2024-12-31
Karen Hill - Electrical Engineer	Lighting Consultant	2024-09-15	2024-11-15
Bob Johnson - Electrical Engineer	Power Grid Lead	2024-09-15	2024-12-01
John Doe - Electrical Engineer	Site Supervisor	2024-05-01	2025-10-31

Add New Project

Assign Employee to Project